RoboLongR: Causally-Consistent Rewards Teach Robots to Progressively Master Long-Horizon Manipulation

Anonymous RA-L Submission

***Abstract*— Long-horizon manipulation requires coordinating interdependent subtasks, yet visual progress alone does not establish whether task prerequisites have been satisfied. We propose RoboLongR, an offline-to-online framework targeting causally consistent rewards that assess progress using execution history. Stage-shuffle supervision encourages execution-order sensitivity, recurrent memory preserves evidence of earlier interactions, and two-round reward learning refines trajectory segmentation. For new tasks, these rewards guide stage-conditioned policy refinement from behavior cloning to online reinforcement learning, while sparse terminal feedback triggers reward correction for repeated high-reward failures. Pretrained on demonstrations from 88 RLBench tasks and Open-X data, RoboLongR achieves 31.8% average success on 18 tasks held out from reward pretraining, exceeding the strongest baseline by 9.2 percentage points. Across four real-robot tasks, it achieves 57.5% success, improving over the comparison method by 32.5 percentage points. Reward-level controls show sensitivity to synthetic order perturbations, while ablations assess component contributions to policy improvement. Anonymous project website: https://Robo-LongR.github.io.**

***Index Terms*— Long-horizon manipulation, causally consistent rewards, reinforcement learning, learning from demonstration.**

I. Introduction

Learning diverse long-horizon manipulation tasks requires robots to refine demonstrated behavior through interaction. Demonstrations initialize behavior, while learned visual rewards provide dense feedback for subsequent reinforcement learning (RL) [1]–[5]. When progress assessment transfers across tasks, these rewards can guide policy adaptation to new tasks [3]. Learning transferable and reliable rewards therefore offers a route to expanding a robot's task repertoire. However, visual progress can be misleading in long-horizon tasks, where actions depend on earlier interactions [6]. Storing an object in a drawer requires opening the drawer, placing the object inside, and closing it. Yet closing an empty drawer can produce a goal-like image without completing the task (Fig. 1 (left)). Rewarding this apparent progress may reinforce behavior that skips essential prerequisites. Such errors become particularly consequential when online exploration takes policies beyond their demonstrations [7], [8]. Reliable rewards must therefore assess progress in light of the execution history.

We define *causal consistency* as agreement between rewarded progress and task prerequisites supported by the execution history. These prerequisites may form a partial order, admitting multiple valid execution sequences. Learning such rewards poses two challenges: successful demonstrations offer little supervision for prerequisite violations, and

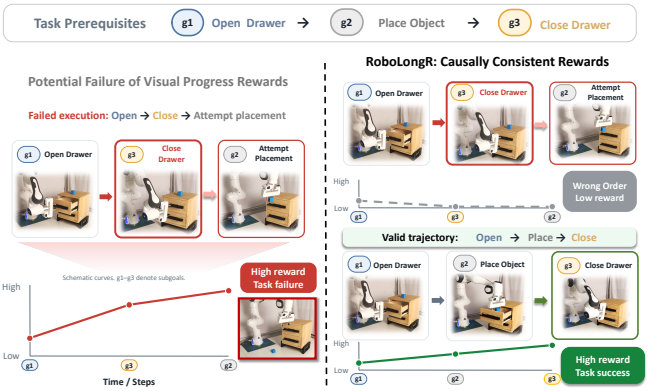


Fig. 1. **Reward guidance beyond visual progress.** Closing an empty drawer can produce a goal-like observation without completing the task. RoboLongR uses stage-order supervision and execution history to encourage prerequisite-sensitive progress estimates. Labels g_1–g_3 denote semantic subgoals. Curves illustrate intended behavior, with reward responses on real executions evaluated separately.

online rollouts on new tasks can expose reward errors absent from training. This motivates our central question: *How can successful demonstrations and sparse interaction feedback yield rewards that guide policy adaptation to new tasks while remaining sensitive to task prerequisites?*

We propose **RoboLongR**, an offline-to-online framework that targets causally consistent rewards for progressive policy refinement (Fig. 2 (right)). From multitask demonstrations, we learn a language-conditioned reward over observation histories. Stage-shuffle reorders intact demonstration segments, using order supervision as a proxy for prerequisite sensitivity. Recurrent memory retains interaction evidence across temporal windows, and a second training round adapts the segments to predicted progress. For downstream tasks, demonstrations initialize the policy through behavior cloning (BC), and the learned reward guides subsequent RL. Separate coarse stages condition BC and scheduled online learning. All RL episodes start from ordinary initial states. Finally, sparse terminal feedback flags repeated high-reward failures and triggers reward correction, linking policy experience to the reward used for further learning.

We pretrain the reward model on demonstrations from 88 RLBench [9] tasks and additional Open-X data [10], then evaluate on 18 disjoint long-horizon RLBench tasks. Downstream tasks provide demonstrations for BC and interaction feedback for policy refinement and reward correction. This setting evaluates reward-guided adaptation beyond the RLBench pretraining tasks. RoboLongR achieves 31.8% average success, versus 7.8% for BC alone, and exceeds

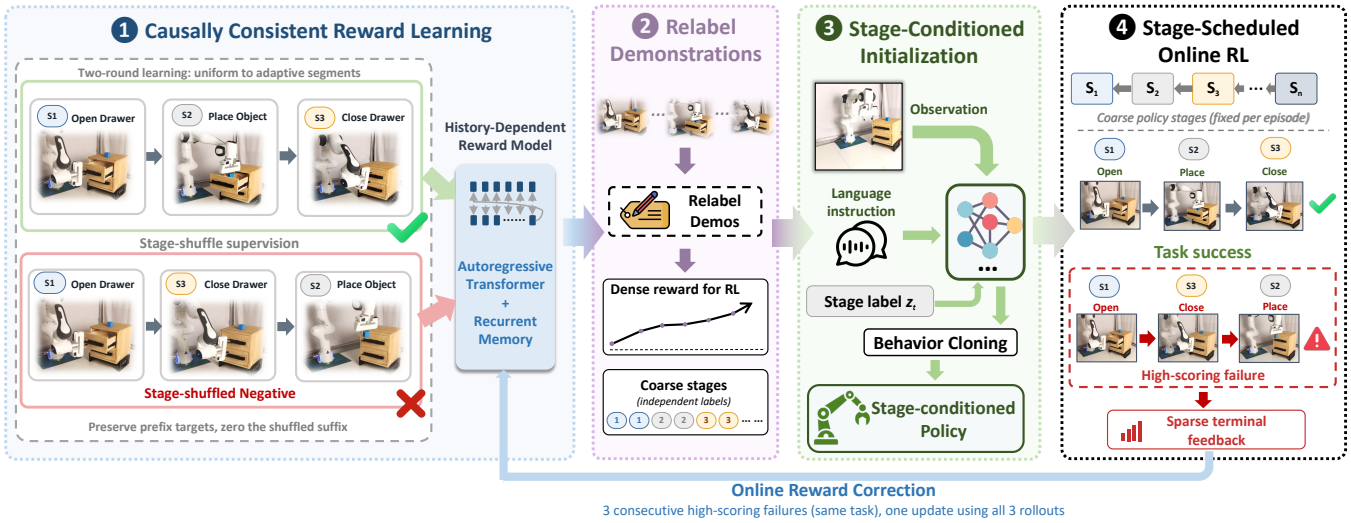


Fig. 2. **RoboLongR learns rewards to guide downstream policy adaptation.** Stage-shuffle uses reward-adaptive segments for order supervision, while separate coarse stages condition the policy. Target-task demonstrations initialize behavior, and dense rewards guide online RL. Sparse terminal feedback triggers reward correction for repeated high-scoring failures.

the strongest baseline by 9.2 percentage points. On four real-robot tasks, it achieves 57.5% success, exceeding the comparison method by 32.5 percentage points.

Our contributions are: (1) stage-order supervision and recurrent memory for learning progress rewards sensitive to task prerequisites; (2) a reward-guided BC-to-RL framework that applies multitask reward pretraining to downstream policy adaptation and corrects rewards using sparse interaction feedback; and (3) evaluations on held-out RLBench tasks and real robots, with reward-level controls and ablations examining order sensitivity and downstream utility.

II. Related Work

Rewards for Robot Learning. Learning rewards from demonstrations has long been studied through inverse reinforcement learning and adversarial imitation learning [11], [12]. Visual and language representations enable progress, value, and similarity signals for robot control, as in LIV [1], VLC [13], GVL [14], and RoboCLIP [2]. ReWiND [3] learns transferable progress rewards from rewound trajectories to guide offline-to-online policy adaptation. General-purpose models further explore trajectory comparisons [15], outcome assessment [16], and dense process rewards [17]. However, progress estimation alone does not establish whether the execution history supports task prerequisites. RoboLongR therefore targets causal consistency through stage-order supervision as a proxy for prerequisite sensitivity, supported by recurrent memory.

Stage-Aware Rewards and Policy Refinement. Stage modeling structures reward estimation and policy improvement for long-horizon tasks. SARM [18] jointly predicts task stages and fine-grained progress, while SARM2 [19] extends stage-aware rewards to multiple tasks and policy improvement from autonomous rollouts. RoboHorizon [20] combines LLM-generated subtask rewards with a multi-view world model. Interaction-based policy refinement [4], [5], [21]–[23] further motivates rewards that remain reliable as policy behavior evolves. RARM [24] addresses unreliable progress estimates through confidence gating. RoboLongR uses adaptive segments to construct order supervision and separate coarse stages for policy conditioning. Within its BC-to-RL pipeline, sparse terminal feedback flags repeated high-scoring failures and triggers online reward correction, allowing interaction outcomes to inform the rewards used for subsequent policy updates.

III. Method

RoboLongR connects *causally consistent reward learning*, *demonstration relabeling*, *stage-conditioned initialization*, and *stage-scheduled online RL* (Fig. 2). Stage-order supervision changes progress targets by reordering intact demonstration segments, while recurrent memory retains evidence of earlier interactions. A second training round adapts reward-learning segments to predicted progress. Separate coarse labels support BC initialization and policy conditioning. The learned reward guides full-task RL, while sparse terminal feedback triggers online reward correction when failed executions repeatedly receive high scores.

A. Problem Formulation

Given a task instruction ℓ, an execution is denoted by $\tau = \{(o_t, s_t, a_t)\}_{t=1}^{T}$, where o_t, s_t, and a_t are RGB observations, robot states, and actions. The environment provides binary terminal success $b(\tau)$. We learn a dense progress reward $R_\phi(o_{1:t}, \ell) \in [0,1]$ to guide policy optimization.

We define *causal consistency* as agreement between rewarded progress and the task prerequisites supported by execution history. Prerequisites can form a partial order: independent placements may be exchanged, whereas placing an arch requires both supports. With no annotated prerequisite graph, we pursue this objective through order-sensitive learning from successful demonstrations.

B. Causally Consistent Reward Learning

Stage-shuffle supervision. Successful demonstrations provide examples of task progress but little direct supervision for misleading progress. We partition each demonstration into contiguous segments $S_1, \ldots, S_{K_a(\tau)}$ and construct a shuffled counterpart with probability $p = 0.5$. For three consecutive segments, stage-shuffle replaces $S_i \to S_{i+1} \to S_{i+2}$ with $S_i \to S_{i+2} \to S_{i+1}$. The perturbation preserves each segment's visual content and internal ordering while changing the context in which later segments appear. The resulting supervision encourages the reward to distinguish locally familiar behavior from progress supported by the execution history.

Let t_* be the first displaced step. The shuffled target preserves the positive target before t_* and is zero thereafter:

$$\tilde{y}_t = \begin{cases} y_t, & t < t_*, \\ 0, & t \geq t_*. \end{cases} \tag{1}$$

For a positive trajectory and a sampled shuffled counterpart, we minimize

$$\mathscr{L}_R = \frac{1}{|\mathscr{M}|} \sum_{t \in \mathscr{M}} \Big[\big(R_\phi(o_{1:t}, \ell) - y_t\big)^2 + \big(R_\phi(\tilde{o}_{1:t}, \ell) - \tilde{y}_t\big)^2 \Big], \tag{2}$$

where $\mathscr{M}$ excludes padding and $\tilde{o}_{1:t}$ is the shuffled observation prefix. Positive regression also applies when no shuffled counterpart is sampled.

This perturbation preserves within-segment motion while changing the order of larger behaviors. For example, an opening–closing–placement permutation contains familiar drawer interactions but violates their demonstrated ordering. To discourage reliance on splice boundaries, we also construct order-preserving positive splices by replacing a segment with the corresponding stage from another successful demonstration. Their boundaries are not exactly matched to those of shuffled trajectories; Sec. IV-C reports reward-level controls. The zero suffix is a training prior for prerequisite sensitivity, not a semantic failure label, and may penalize valid alternative orders or recoveries.

Recurrent execution memory. Order supervision requires access to earlier observations, including evidence that may no longer be visible. Two RGB views are encoded by frozen DINOv2 ViT-B/14 [25], and the instruction by frozen all-MiniLM-L6-v2. Projected visual and language features feed an autoregressive Transformer, followed by a recurrent module and an MLP progress head. Attention spans 100 frames; longer sequences use causal chunks with recurrent state carried across chunks, detached during training, and reset at episode boundaries. Each prediction therefore uses only the observed prefix. Memory preserves evidence beyond the attention window, while stage-shuffle supplies supervision that encourages its use in order-sensitive progress estimation.

Reward-adaptive segmentation. The segments determine which temporal relationships stage-shuffle perturbs. We therefore refine them using the reward's own predicted progress in a second training round (Algorithm 1). Round 1 uses uniform temporal partitions and targets $y_t = (t/T)^2$.

Algorithm 1 Two-round reward learning with adaptive stage-shuffle

Require: demonstrations $\mathscr{D}$, instructions ℓ, shuffle rate p
initialize R_ϕ and set $y_t = (t/T)^2$
for $q = 1, 2$ **do**
  if $q = 1$ **then**
    partition each trajectory uniformly
  else
    predict progress with the first-round checkpoint
    min-max normalize predictions to obtain targets
    smooth predicted curves and compute adaptive partitions
  end if
  for each minibatch $B \subset \mathscr{D}$ **do**
    construct order-preserving positive splices
    sample shuffled counterparts with probability p
    set shuffled targets to zero from the first displaced step
    update ϕ using masked positive regression and sampled negative regression
  end for
end for
return R_ϕ

Round 2 continues from the first-round checkpoint, using its predictions as targets after per-demonstration min-max normalization.

To locate segment boundaries, we smooth each predicted curve with an 11-frame, degree-three Savitzky–Golay filter. We select local maxima of the absolute second derivative that exceed one standard deviation of curvature magnitude, with boundaries at least five frames apart. Trajectories without a sufficiently strong change point use five uniform segments; stage-shuffle requires at least three segments. This procedure adapts the perturbation units to changes in predicted progress. The resulting segments remain trajectory-specific temporal partitions and need not coincide with semantic subtasks.

C. Demonstration Relabeling

We annotate target-task demonstration prefixes with learned progress scores $R_\phi(o_{1:t}, \ell)$ and assign separate coarse stage labels for policy learning. Reward augmentation and policy conditioning use separate temporal representations. Adaptive reward segments vary across demonstrations, whereas policy stages must retain a shared meaning. We therefore condition the policy on coarse labels $z_t \in \{1, \ldots, K_c\}$ from task-category templates, such as reach, grasp, lift, transport, and place. Gripper transitions, object motion, contact, and release determine their boundaries. This separation combines trajectory-specific order supervision with a consistent conditioning vocabulary; segmentation and stage-assignment ablations examine this design.

D. Stage-Conditioned Initialization

The actor and critic receive joint positions and velocities, end-effector pose, gripper opening, a frozen language embedding $\mathbf{e}_\ell$, and a learned stage embedding. Actions comprise an end-effector displacement, quaternion rotation, and gripper command. We initialize $\pi_\theta(s_t, \mathbf{e}_\ell, z_t)$ with 50 epochs of BC using per-step coarse labels.

E. Stage-Scheduled Online RL

Following BC initialization, PPO [26] optimizes the raw learned progress reward to adapt the policy through interaction:

$$r_t^{\mathrm{RL}} = R_\phi(o_{1:t}, \ell). \tag{3}$$

Terminal success controls stage scheduling and reward correction and is not added to the PPO reward. During RL, the stage input remains fixed within each episode and follows a backward schedule from the final stage to the first, advancing when the task success rate exceeds 0.8. Every episode starts from the ordinary initial-state distribution and attempts the full task. Evaluation fixes the stage input to the first stage. The schedule thus changes policy conditioning during full-task learning without requiring mid-task resets.

Online reward correction. Online interaction exposes execution histories beyond the demonstrations used to learn the reward. We use sparse terminal outcomes to flag recurring disagreements between predicted progress and task completion. A rollout is flagged when $b(\tau) = 0$ and $\max_t R_\phi(o_{1:t}, \ell) > 0.65$. Three consecutive flagged episodes form a task-specific correction batch $\mathscr{B}^- = \{\tau_j^-\}_{j=1}^3$. We average the per-trajectory losses over all three rollouts:

$$\mathscr{L}_{\mathrm{corr}} = \frac{1}{3}\sum_{j=1}^{3}\frac{1}{T_j}\sum_{t=1}^{T_j} R_\phi(o_{1:t}^{(j)}, \ell)^2, \tag{4}$$

where T_j and $o_{1:t}^{(j)}$ denote the length and observation prefix of τ_j^-. A single Adam step on this joint loss updates the Transformer, recurrent module, and progress head, using learning rate 10^{-4} and gradient norm clipped to 1.0. Both encoders remain frozen. After the update, the correction batch is cleared and the trigger counter is reset; no offline replay is included. The correction requires no new per-step labels. Because a failed rollout can contain valid partial progress, the trigger identifies candidate reward errors, and whole-trajectory correction may also suppress valid prefixes. This feedback connects policy experience to subsequent reward updates: the reward guides policy refinement, and recurring high-scoring failures prompt revision of that guidance.

Implementation details. Mean-pooled visual features are 768-dimensional, and language features are 384-dimensional; both are projected to dimension 512. Images use 224×224 crops with DINOv2 normalization. The Transformer has four layers, eight heads, feed-forward dimension 2048, and dropout 0.1; the recurrent hidden size is 512. Reward learning uses Adam with learning rate 10^{-4}, batch size 8, and 60 epochs. PPO uses learning rate 3×10^{-4}, clip ratio 0.2, discount 0.99, GAE 0.95, 2,048-step rollouts, and ten optimization epochs per update. Interaction budgets are 2×10^5 steps per simulation task and 120 episodes per real-robot task at 10 Hz, with a 500-step horizon. Experiments use seeds 0, 1, and 2 on eight RTX 4090 GPUs.

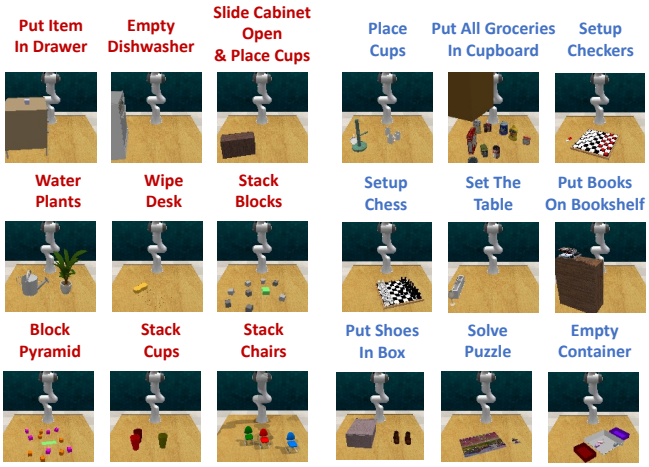


Fig. 3. **RLBench evaluation tasks.** The 18 long-horizon tasks are excluded from offline reward pretraining.

IV. Experiments

We evaluate RoboLongR in simulation and on real robots to examine reward-guided policy improvement and the sensitivity of learned rewards to execution history. Our experiments address four questions: (1) Can the learned rewards guide online adaptation and improve success on tasks excluded from reward pretraining (Sec. IV-B)? (2) Can the reward model distinguish demonstrated executions from synthetic stage-order perturbations (Sec. IV-C)? (3) How do stage-shuffle supervision, recurrent memory, stage scheduling, and reward correction contribute to policy improvement (Sec. IV-D)? (4) Do the policy gains extend to real-robot manipulation, and how do learned rewards respond to executions that violate task prerequisites (Sec. IV-E)?

A. Experimental Setup

Benchmark and data. Our RLBench [9] protocol separates 88 reward-pretraining tasks from 18 long-horizon evaluation tasks (Fig. 3). The pretraining split provides 20 successful demonstrations per task, totaling 1,760 trajectories. Reward pretraining additionally uses 4,000 trajectories from the bridge and fractal20220817 subsets of Open-X Embodiment [10], comprising 25% of each batch. Open-X supplies progress supervision and rewound negatives, while stage-shuffle uses segmented RLBench demonstrations. ReWiND is retrained on the same data mixture.

Each evaluation task supplies 20 additional demonstrations for reward relabeling and BC initialization, followed by interaction for policy refinement and reward correction. This protocol evaluates reward-guided adaptation to tasks held out from reward pretraining. The benchmark contains nine strong-dependency tasks with prominent physical prerequisites and nine weak-dependency tasks allowing flexible object order. Drawer storage, for example, requires opening

TABLE I

SUCCESS RATES (%) ON 18 LONG-HORIZON RLBENCH TASKS GROUPED BY DEPENDENCY STRENGTH. RESULTS REPORT MEAN ± SAMPLE STANDARD DEVIATION OVER THREE SEEDS, WITH 50 TRIALS PER SEED. AVG. DENOTES THE WITHIN-GROUP MEAN.

Strong-dependency tasks with prominent physical prerequisites										
Models	Avg. (%) ↑	Put Item In Drawer	Empty Dishwasher	Slide Cabinet Open & Place Cups	Water Plants	Wipe Desk	Block Pyramid	Stack Blocks	Stack Cups	Stack Chairs
BC-only	8.2	14.0±3.5	9.3±2.3	6.7±3.1	16.7±3.1	20.0±2.0	0.0	2.7±1.2	4.7±4.2	0.0
Sparse-PPO	7.1	12.7±2.3	8.0±3.5	5.3±2.3	14.7±4.2	18.0±3.5	0.0	2.0±2.0	3.3±3.1	0.0
LIV	3.5	6.7±4.2	4.0±3.5	1.3±2.3	8.0±3.5	11.3±1.2	0.0	0.0	0.0	0.0
Progress+Mono	20.9	30.7±3.1	24.0±2.0	20.0±2.0	33.3±4.2	38.0±2.0	8.7±3.1	11.3±4.2	15.3±3.1	6.7±4.2
ReWiND	23.6	34.0±3.5	27.3±2.3	22.7±4.2	36.7±4.2	41.3±1.2	10.0±2.0	14.0±2.0	17.3±4.2	8.7±2.3
RoboLongR	**37.8**	**49.3**±2.3	**42.0**±2.0	**36.7**±2.3	**51.3**±2.3	**55.3**±3.1	**22.7**±3.1	**27.3**±4.2	**32.0**±3.5	**23.3**±4.2

Weak-dependency tasks allowing flexible subtask order										
Models	Avg. (%) ↑	Place Cups	Put All Groceries In Cupboard	Setup Checkers	Setup Chess	Set The Table	Put Books On Bookshelf	Put Shoes In Box	Solve Puzzle	Empty Container
BC-only	7.4	16.0±4.0	2.0±3.5	3.3±1.2	0.0	0.0	12.7±4.2	10.0±4.0	0.0	22.7±4.2
Sparse-PPO	6.5	14.0±3.5	1.3±1.2	2.7±2.3	0.0	0.0	11.3±3.1	8.7±2.3	0.0	20.7±2.3
LIV	2.4	6.0±4.0	0.0	0.0	0.0	0.0	4.7±1.2	4.0±3.5	0.0	6.7±2.3
Progress+Mono	19.3	30.7±2.3	10.7±3.1	13.3±4.2	6.7±1.2	5.3±3.1	26.0±2.0	22.7±3.1	7.3±2.3	50.7±4.2
ReWiND	21.6	34.0±2.0	12.7±4.2	15.3±2.3	8.0±2.0	6.7±3.1	29.3±1.2	26.0±2.0	8.7±2.3	54.0±3.5
RoboLongR	**25.9**	**38.0**±2.0	**16.7**±4.2	**19.3**±2.3	**11.3**±4.2	**10.0**±4.0	**33.3**±2.3	**34.0**±2.0	**12.7**±3.1	**57.3**±3.1

TABLE II

REWARD-LEVEL DIAGNOSTICS ON HELD-OUT TRAJECTORIES. Δ DENOTES THE MEAN REWARD DIFFERENCE BETWEEN CORRECT AND PERTURBED TRAJECTORIES.

	ReWiND	RoboLongR
Δ(order-perturbed) ↑	0.04	**0.51**
Δ(rewound) ↑	0.38	0.42
Δ(spliced) ↓	0.03	0.06
Ranking accuracy (%) ↑	54.2	**91.7**
Progress MSE ↓	0.021	0.023

before placement, whereas table setting permits independent placements. Both groups can contain partial orders. The grouping examines whether gains differ with dependency strength, while component attribution is addressed separately by ablation.

Baselines. All RL comparisons share demonstrations, stage-conditioned policy architecture, BC initialization, and interaction budgets. *BC-only* measures performance before online refinement, and Sparse-PPO uses binary terminal feedback. *LIV* [1] provides pretrained visual-language rewards without task-specific reward training. Scores are min-max normalized per rollout before PPO updates. *ReWiND* [3] is the closest progress-reward baseline and uses rewound negatives. *Progress+Mono* is an architecture-matched stage-aware control: it replaces stage-shuffle with a stage-prediction head and a penalty on decreasing predicted stage indices. Its objective combines positive progress regression, stage cross-entropy, and an expected-stage monotonicity penalty, with unit weights on both auxiliary terms.

Evaluation protocol. Simulation evaluation uses 50 episodes per task per seed with randomized layouts and three seeds, totaling 150 trials per task. We report mean success and sample standard deviation across seeds. Benchmark averages weight tasks equally.

B. Policy Improvement on RLBench

Across the 18 tasks excluded from reward pretraining, RoboLongR achieves 31.8% mean success, compared with 7.8% for BC-only and 22.6% for ReWiND (Table I). The 9.2-percentage-point gain over ReWiND shows improved downstream adaptation under the shared demonstration and interaction budgets. Sparse-PPO, LIV, and Progress+Mono achieve 6.8%, 2.9%, and 20.1%, respectively.

Dependence on task structure. RoboLongR exceeds ReWiND on all 18 task means. The group averages are 37.8% versus 23.6% on strong-dependency tasks and 25.9% versus 21.6% on weak-dependency tasks. The corresponding gains are 14.2 and 4.2 percentage points, computed before rounding. The larger gain on strong-dependency tasks is consistent with the motivation for history-sensitive rewards, although these groups also differ in manipulation demands.

Improvement beyond initialization. The gains over BC-only show the benefit of reward-guided interaction. Sparse-PPO remains near the BC reference, while Progress+Mono and ReWiND improve success. Progress+Mono's lower performance than RoboLongR suggests that stage prediction and monotonicity alone do not reproduce the gains. Component ablations follow in Sec. IV-D.

The gains also vary in absolute difficulty. RoboLongR reaches 49.3% on Put Item In Drawer and 22.7% on Block Pyramid, compared with 34.0% and 10.0% for ReWiND. Setup Chess and Set The Table remain challenging at 11.3% and 10.0%. Thus, improved reward guidance supports broader task adaptation while substantial execution difficulty remains.

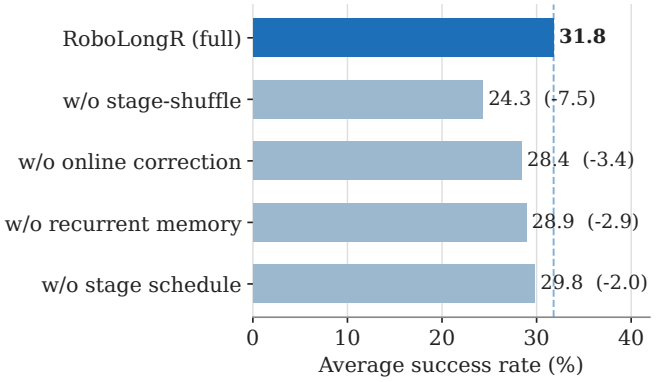


Fig. 4. **Component ablations on RLBench.** Mean task success after removing individual components under shared demonstrations and interaction budgets.

C. Execution-Order Sensitivity

Reward-level protocol. We evaluate the reward independently of policy performance on held-out demonstrations. Correct trajectories preserve the demonstrated order. Order-perturbed trajectories swap intact stage segments, rewound trajectories introduce regression-style corruption, and order-preserving splices replace a segment with the corresponding stage from another successful demonstration. We measure $\Delta(x) = \bar{r}(\text{correct}) - \bar{r}(x)$, where $\bar{r}(\tau) = T^{-1}\sum_t R_\phi(o_{1:t}, \ell)$. We report pairwise ranking accuracy and progress MSE against the time-based target $(t/T)^2$ on correct demonstrations.

Sensitivity and controls. Table II shows an order-perturbation margin of 0.51 for RoboLongR versus 0.04 for ReWiND, with ranking accuracy of 91.7% versus 54.2%. Progress MSE remains similar (0.023 versus 0.021). The margins on rewound trajectories are closer (0.42 versus 0.38), whereas RoboLongR's margin on order-preserving splices is only 0.06. This contrast supports sensitivity to the tested order changes beyond a general penalty on spliced sequences.

Training swaps the latter two of three consecutive segments. Evaluation broadens this construction to adjacent and more distant pairs: RoboLongR's margins are 0.42, 0.53, and 0.61 for adjacent, medium, and distant swaps, compared with 0.03, 0.04, and 0.06 for ReWiND. The evaluation therefore varies swap distance but is not uniformly disjoint from the training corruption family. Valid orders present in held-out weak-dependency demonstrations remain in the correct set. These controls demonstrate sensitivity to the tested perturbations. Unmatched boundary statistics and untested valid orders limit conclusions about semantic prerequisite satisfaction.

D. Ablation Studies

Component contributions. Fig. 4 compares component removals under shared demonstrations and interaction budgets. Removing stage-shuffle reduces mean success from 31.8% to 24.3%, the largest observed drop. Removing online correction, recurrent memory, or the stage schedule yields 28.4%, 28.9%, and 29.8%, respectively.

Reward refinement and stage representations. One, two, and three reward-training rounds achieve 28.8%, 31.8%, and 32.0% success, accompanied by reward margins of 0.38, 0.51, and 0.52. Two rounds capture most of the observed improvement in both measures. Replacing adaptive shuffle segments with coarse segments yields 29.8%, while using adaptive indices for both reward augmentation and policy conditioning yields 30.4%. These comparisons favor refining the reward-training partitions while retaining a separate, consistent vocabulary for policy stages.

Adaptive partition behavior. Curvature-based boundaries are detected in 91% of opening-and-closing demonstrations,

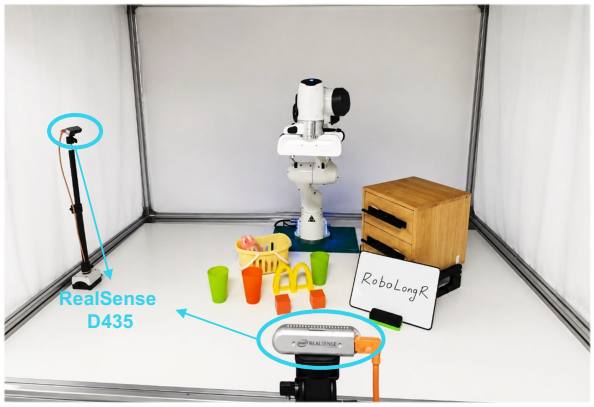


Fig. 5. **Real-robot platform.** A Franka Panda with front and right RealSense D435 cameras.

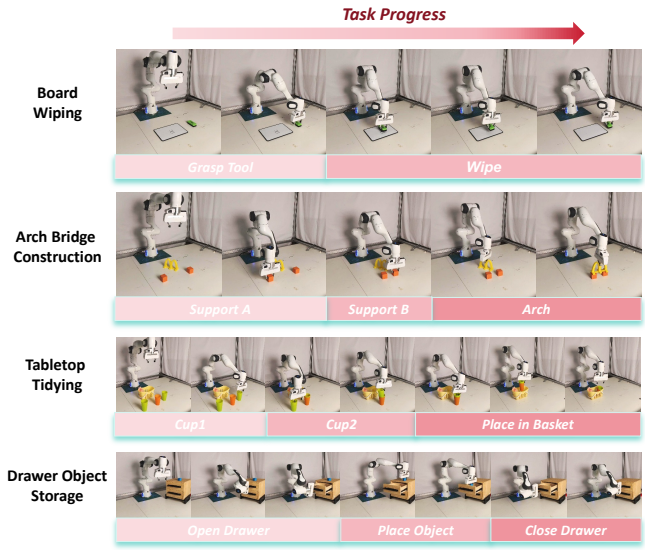


Fig. 6. **Four real-robot tasks and their representative subtask sequences.**

TABLE III

REAL-ROBOT SUCCESS RATES (%). TEN TRIALS PER TASK AND METHOD. BRACKETS GIVE WILSON 95% INTERVALS.

Task	ReWiND	RoboLongR
Drawer Object Storage	20 [5.7, 51.0]	**70** [39.7, 89.2]
Board Wiping	50 [23.7, 76.3]	**70** [39.7, 89.2]
Tabletop Tidying	20 [5.7, 51.0]	**50** [23.7, 76.3]
Arch Bridge Construction	10 [1.8, 40.4]	**40** [16.8, 68.7]
Pooled (40 trials)	25.0 [14.2, 40.2]	**57.5** [42.2, 71.4]

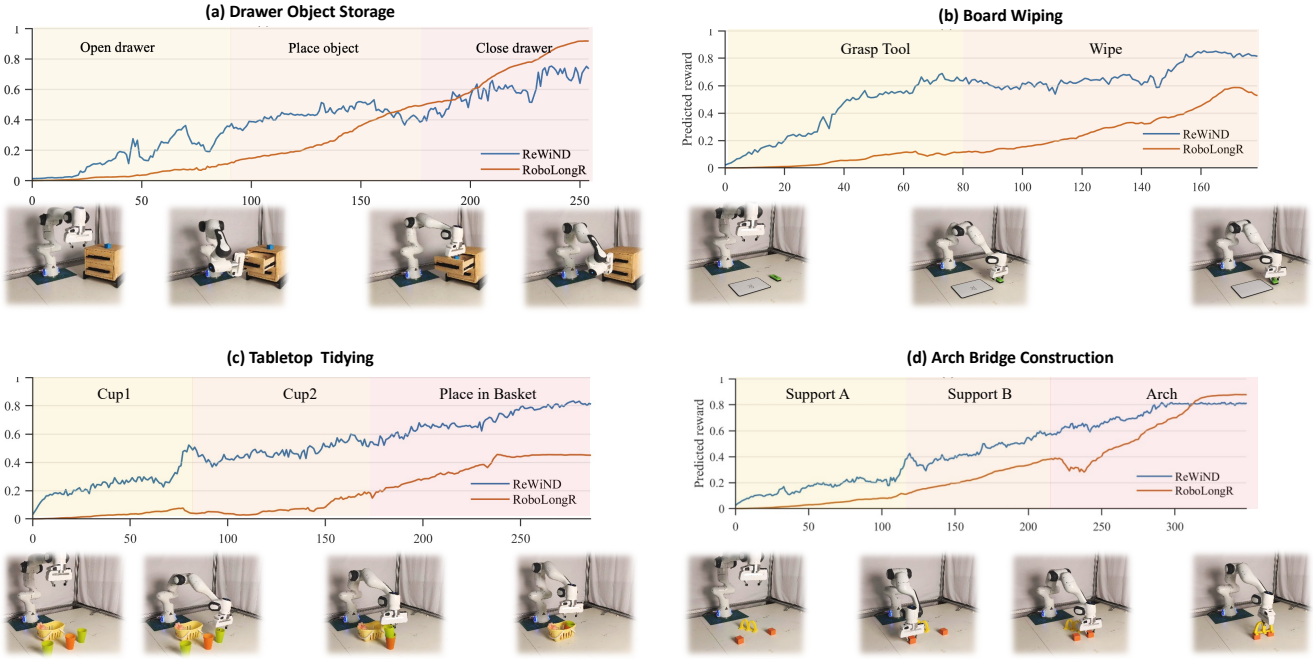


Fig. 7. **Reward progress on valid real-robot executions.** Shaded regions indicate semantic task stages, and images show corresponding observations. Raw reward scales are model-specific.

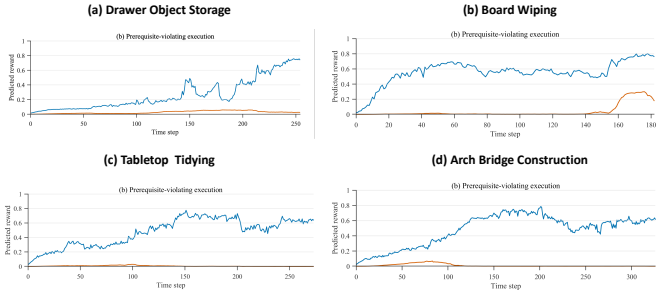


Fig. 8. **Reward responses to prerequisite violations.** ReWiND (blue) and RoboLongR (orange) rewards on held-out failed rollouts containing prerequisite violations. Valid executions are shown in Fig. 7.

78% of pick-and-place, 84% of insertion and alignment, and 52% of composition. The corresponding mean adaptive segment counts are 3.8, 4.6, 4.1, and 6.2. Repeated object manipulations can yield smoother progress curves, making the five-segment fallback useful for composition.

E. Real-Robot Validation

Experimental setup. We use a seven-degree-of-freedom Franka Panda with front and right Intel RealSense D435 cameras (Fig. 5). Four tasks span articulated-object interaction, tool use, multi-object manipulation, and assembly (Fig. 6). Drawer Object Storage requires opening the drawer, inserting the object, and closing it. Board Wiping requires retrieving a tool before cleaning. Tabletop Tidying requires stacking three cups before transferring them into a basket. Arch Bridge Construction requires positioning two supports before placing the arch.

We compare RoboLongR with ReWiND, the strongest simulation baseline. Both methods use the same ten successful GELLO [27] demonstrations per task for reward learning and BC initialization, followed by online refinement under the same interaction budget. Each method is evaluated in ten trials per task with randomized placements. Success requires an object inside the fully closed drawer, a clean marked region with the tool released, a stable cup stack inside the basket, or an arch resting on upright supports, respectively. One annotator blind to the method judges complete-task success from video, and failed trials are not selectively repeated.

Policy improvement. Table III reports 23/40 successes for RoboLongR and 10/40 for ReWiND, corresponding to pooled success rates of 57.5% and 25.0%, a gain of 32.5 percentage points. RoboLongR achieves higher observed success on all four tasks, consistent with the simulation results. With ten trials per task, the Wilson 95% intervals remain wide. The pooled interval summarizes this fixed task mixture and does not establish per-task statistical significance.

Reward analysis. We examine reward behavior using separately recorded held-out rollouts excluded from reward training, BC, policy optimization, and model selection. We compare each model's responses on valid executions (Fig. 7) with those on failed executions containing prerequisite violations (Fig. 8).

On valid executions, both methods track task progress, although RoboLongR produces lower raw scores on some tasks. Absolute scores are not directly comparable across independently trained models, so we assess the separation between valid and failed executions within each model. The two figures show that ReWiND assigns substantial rewards to both valid and failed executions in these examples, whereas RoboLongR exhibits clearer separation, particularly

on Drawer Object Storage, Tabletop Tidying, and Arch Bridge Construction. On Board Wiping, RoboLongR's score rises modestly near the end of the failed execution as the scene resembles completed wiping, but remains below its score for the valid execution.

These comparisons concern executions that ultimately fail. In particular, the negative drawer rollout fails to store the object, whereas a premature attempt followed by successful recovery need not imply task failure. These qualitative examples do not exhaust alternative valid orders or recoveries, nor isolate the contribution of stage-shuffle from other components.

V. Conclusion and Discussion

We present **RoboLongR**, an offline-to-online framework for history-dependent reward learning that targets causal consistency. RoboLongR combines stage-order supervision and recurrent memory with stage-conditioned policy refinement and online reward correction. Following reward pretraining on 88 RLBench tasks, evaluations on 18 long-horizon tasks excluded from pretraining and four real-robot tasks demonstrate improved complete-task success over the tested baselines. Reward-level controls show sensitivity to synthetic order perturbations, while removing stage-shuffle yields the largest success-rate drop among the ablations.

Several limitations remain. Synthetic shuffles may introduce boundary shortcuts, while their targets can penalize valid alternative orders or recoveries. Whole-rollout correction may suppress valid prefixes and cause reward drift. The framework relies on demonstrations, engineered stage cues, and terminal feedback. Future work will explore localized reward correction, evaluation of valid execution orders and recoveries, and stability over extended policy improvement.

References

[1] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman, "Liv: Language-image representations and rewards for robotic control," in *International Conference on Machine Learning (ICML)*, 2023.

[2] S. A. Sontakke, J. Zhang, S. M. R. Arnold, K. Pertsch, E. Bıyık, D. Sadigh, C. Finn, and L. Itti, "Roboclip: One demonstration is enough to learn robot policies," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[3] J. Zhang, Y. Luo, A. Anwar, S. A. Sontakke, J. J. Lim, J. Thomason, E. Bıyık, and J. Zhang, "Rewind: Language-guided rewards teach robot policies without new demonstrations," in *Proceedings of the 9th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 460–488.

[4] J. Luo, C. Xu, J. Wu, and S. Levine, "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning," *arXiv preprint arXiv:2410.21845*, 2025.

[5] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu, "RL-100: Performant robotic manipulation with real-world reinforcement learning," *arXiv preprint arXiv:2510.14830*, 2025.

[6] Z. Chen, J. Yin, Y. Chen, J. Huo, P. Tian, J. Shi, Y. Hou, Y. Li, and Y. Gao, "DeCo: Task decomposition and skill composition for zero-shot generalization in long-horizon 3D manipulation," *IEEE Robotics and Automation Letters*, 2026.

[7] A. Pan, K. Bhatia, and J. Steinhardt, "The effects of reward misspecification: Mapping and mitigating misaligned models," in *International Conference on Learning Representations (ICLR)*, 2022.

[8] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward hacking," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[9] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "Rlbench: The robot learning benchmark and learning environment," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.

[10] A. O'Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, *et al.*, "Open x-embodiment: Robotic learning datasets and RT-X models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.

[11] P. Abbeel and A. Y. Ng, "Apprenticeship learning via inverse reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2004.

[12] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[13] M. Alakuijala, R. McLean, I. Woungang, N. Farsad, S. Kaski, P. Marttinen, and K. Yuan, "Video-language critic: Transferable reward functions for language-conditioned robotics," *arXiv preprint arXiv:2405.19988*, 2024.

[14] Y. J. Ma, J. Hejna, C. Fu, D. Shah, J. Liang, Z. Xu, S. Kirmani, P. Xu, D. Driess, T. Xiao, *et al.*, "Vision language models are in-context value learners," in *International Conference on Learning Representations (ICLR)*, 2025.

[15] A. Liang, Y. Korkmaz, J. Zhang, M. Hwang, A. Anwar, S. Kaushik, A. Shah, A. S. Huang, L. Zettlemoyer, D. Fox, Y. Xiang, A. Li, A. Bobu, A. Gupta, S. Tu, E. Bıyık, and J. Zhang, "Robometer: Scaling general-purpose robotic reward models via trajectory comparisons," in *Robotics: Science and Systems (RSS)*, 2026, arXiv:2603.02115.

[16] T. Lee, A. Wagenmaker, K. Pertsch, P. Liang, S. Levine, and C. Finn, "Roboreward: General-purpose vision-language reward models for robotics," *arXiv preprint arXiv:2601.00675*, 2026.

[17] S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang, "A vision-language-action-critic model for robotic real-world reinforcement learning," *arXiv preprint arXiv:2509.15937*, 2025.

[18] Q. Chen, J. Yu, M. Schwager, P. Abbeel, Y. Shentu, and P. Wu, "Sarm: Stage-aware reward modeling for long horizon robot manipulation," in *International Conference on Learning Representations (ICLR)*, 2026.

[19] Q. Chen, H. Zheng, J. Yu, S. Huang, J. Sun, K. Goldberg, C. Wen, P. Abbeel, Y. Shentu, P. Wu, and M. Schwager, "Sarm2: Multi-task stage aware reward modeling for self improving robotic manipulation," *arXiv preprint arXiv:2606.10305*, 2026.

[20] Z. Chen, J. Huo, Y. Chen, and Y. Gao, "Robohorizon: An llm-assisted multi-view world model for long-horizon robotic manipulation," *arXiv preprint arXiv:2501.06605*, 2025.

[21] J. Yang, K. Lin, J. Li, W. Zhang, T. Lin, L. Wu, Z. Su, H. Zhao, Y.-Q. Zhang, L. Chen, P. Luo, X. Yue, and H. Li, "Rise: Self-improving robot policy with compositional world model," *arXiv preprint arXiv:2602.11075*, 2026.

[22] Y. Liu, H. Li, S. Tian, Y. Qin, Y. Chen, Y. Zheng, Y. Huang, and D. Zhao, "Towards long-lived robots: Continual learning vla models via reinforcement fine-tuning," *arXiv preprint arXiv:2602.10503*, 2026.

[23] H. Zhang, S. Zhang, J. Jin, Q. Zeng, R. Li, and D. Wang, "Robustvla: Robustness-aware reinforcement post-training for vision-language-action models," *arXiv preprint arXiv:2511.01331*, 2025.

[24] P. Yang, X. Wang, P. Jing, K. Wen, Y. Qu, Z. Huang, M. Fu, X. Liu, Y. Shen, and F. Shi, "Rarm: Confidence-gated progress reward modeling for rl in manipulation," *arXiv preprint arXiv:2606.22027*, 2026.

[25] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "Dinov2: Learning robust visual features without supervision," *Transactions on Machine Learning Research (TMLR)*, 2024.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[27] P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel, "Gello: A general, low-cost, and intuitive teleoperation framework for robot manipulators," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 12 156–12 163.